

Esercizio A1

Limite asintotico inferiore Data la componente non ricorsiva, è facile vedere che $T(n) = \Omega(n^2)$.

Limite asintotico inferiore Supponiamo che $T(n) = O(n^2)$. Dobbiamo dimostrare che

$$\exists c > 0, \exists m > 0 : T(n) \leq cn^2, \forall n \geq m$$

- **Casi base:**

$$T(i) = 1 \leq c \cdot i^2 \Rightarrow c \geq 1/i^2$$

per $1 \leq i \leq 5$; tutte queste disequazioni sono valide per $c \geq 1$.

- **Ipotesi induttiva:**

$$T(k) \leq ck^2, \forall k < n$$

- **Passo induttivo:**

$$\begin{aligned} T(n) &= 3T(\lfloor n/3 \rfloor) + 4T(\lfloor n/4 \rfloor) + 5T(\lfloor n/5 \rfloor) + 6T(\lfloor n/6 \rfloor) + n^2 \\ &\leq 3c\lfloor n/3 \rfloor^2 + 4c\lfloor n/4 \rfloor^2 + 5c\lfloor n/5 \rfloor^2 + 6c\lfloor n/6 \rfloor^2 + n^2 && \text{Ipotesi induttiva} \\ &\leq cn^2/3 + cn^2/4 + cn^2/5 + cn^2/6 + n^2 && \text{Semplificazione limite inferiore} \\ &= \frac{20 + 15 + 12 + 10}{60}cn^2 + n^2 \\ &= 57/60cn^2 + n^2 = 19/20cn^2 + n^2 \leq cn^2 \end{aligned}$$

L'ultima disequazione è vera per $c \geq 20$.

Abbiamo quindi dimostrato che $T(n)$ è $\Theta(n^2)$.

Esercizio A2

Il problema può essere risolto tramite tecnica divide-et-impera. Dato un nodo t , siano t_l e t_r i suoi sottoalberi sinistro e destro di dimensione s_l e s_r (potenzialmente zero, se il sottoalbero è costituito da un puntatore **nil**). Si chiamerà quindi ricorsivamente la procedura sul sottoalbero sinistro, con s_l valori; si piazzerà il valore $s_l + 1$ -esimo nella radice; e quindi si richiamerà la procedura sul sottoalbero destro, con s_r valori.

Poichè questo algoritmo è sostanzialmente una visita, la sua complessità sarà pari a $\Theta(n)$.

```
binaryInsert(TREE T, int[] A)
```

```
    biRec(T, A, 1)
```

```
biRec(TREE t, int[] A, int i)
```

```
    int leftSize = 0
```

```
    if T.left  $\neq$  nil then
```

```
        leftSize = t.left.size
```

```
        biRec(t.left, A, i)
```

```
    t.value = A[i + leftSize]
```

```
    if T.right  $\neq$  nil then
```

```
        biRec(t.right, A, i + leftSize + 1)
```

Esercizio A3

Il primo passo è notare che se esiste un valore di maggioranza, deve essere il valore contenuto nella posizione mediana $\lfloor (n+1)/2 \rfloor$. Se n è dispari, questo è l'elemento centrale. Qualunque maggioranza deve includere questo elemento e altri $(n-1)/2$ elementi; infatti, a sinistra e destra di tale elemento ci stanno esattamente $(n-1)/2$ elementi. Se n è pari, sia $n/2$ che $n/2 + 1$ devono appartenere alla maggioranza, per lo stesso ragionamento di cui sopra.

Dato questo valore v , è sufficiente scrivere una procedura di ricerca dicotomica che assuma l'esistenza di tale valore nel vettore e restituisca l'indice più basso in cui tale valore si presenta. Il problema di trovare l'indice più basso è stato proposto nel compito del 7/2/2019. Qui riporto l'intero codice per completezza, ma sarebbe stato possibile semplicemente invocare la funzione, quindi l'esercizio era risolvibile in quattro righe di codice.

Conoscendo l'indice del primo elemento, è necessario vedere se l'elemento in posizione $A[first + \lfloor n/2 \rfloor]$ è uguale ad esso, nel qual caso esiste una maggioranza di elementi uguali, essendo il vettore ordinato.

```
int hasMajority(int[] A, int n)
    int candidate = A[ $\lfloor (n+1)/2 \rfloor$ ]
    int first = searchFirst(A, 1, n, candidate)
    return A[first] == A[first +  $\lfloor n/2 \rfloor$ ]
```

```
int searchFirst(int[] A, int i, int j, int v)
    if i == j then
        return i
    else
        int m =  $\lfloor (i+j)/2 \rfloor$ 
        if v ≤ A[m] then
            return searchFirst(A, i, m, v)
        else
            return searchFirst(A, m+1, j, v)
```

Il costo computazionale, dato dalla ricerca dicotomica, è $O(\log n)$.

Esercizio B1

Questo esercizio può essere risolto tramite tecnica greedy, con una dimostrazione semplice.

Innanzitutto, anche se non richiesto, notiamo che una sequenza di cifre può dare origine ad un numero palindromo se e solo se esiste al più una cifra il cui numero di occorrenze è dispari. Se esiste esattamente una cifra il cui numero di occorrenze è dispari, andrà al centro.

Dopo di che, la scelta greedy sarà quella di piazzare le cifre più alte all'esterno (quindi all'inizio), per ottenere un numero più grande.

Per prima cosa quindi realizziamo un conteggio del numero delle cifre, simile a quanto fatto con counting sort. Il costo di questa operazione è $O(n)$ (l'inizializzazione di un vettore contenente 10 elementi ha costo costante).

Dopo di che, piazziamo le cifre dall'esterno verso l'interno. Come richiesto, la versione presentata non effettua il controllo di palindromicità. È semplice aggiungere un ciclo su *count* che verifichi che il numero di elementi dispari sia al massimo uno.

Il costo della funzione è $O(n)$. Nel soluzioni proposte dagli studenti, ho visto versioni con costo $O(n \cdot n!)$ (generazione di tutte le permutazioni possibile e verifica di palindromicità), $O(n^3)$ (ricerca quadratica della coppia di valori massimi, scambio con i valori esterni, ripetizione nel sottovettore dei valori interni), $O(n^2)$ (come sopra, ma con ricerca lineare della coppia di valori massimi), $O(n \log n)$ (più simile alla mia versione, ma con ordinamento al posto di un approccio più simile al counting sort).